Hands-On Exercises: Authentication and Authorization in ASP.NET Core Web API Microservices

Question 1: Implement JWT Authentication in ASP.NET Core Web API

appsettings.json

```
{
  "Jwt": {
    "Key": "ThisIsASecretKeyForJwtToken123456789",
    "Issuer": "MyAuthServer",
    "Audience": "MyApiUsers",
    "DurationInMinutes": 60
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

LoginModel.cs

```
namespace JwtAuthenticationDemo.Models
{
    public class LoginModel
    {
        public string Username { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}
```

Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options =>
{
```

```
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
    };
});

builder.Services.AddAuthorization();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

AuthController.cs

```
using JwtAuthenticationDemo.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
```

```
namespace JwtAuthenticationDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [AllowAnonymous]
    public class AuthController : ControllerBase
    {
        [HttpPost("login")]
        public IActionResult Login([FromBody] LoginModel model)
        {
            if (model.Username == "admin" && model.Password == "admin123")
            {
                var token = GenerateJwtToken(model.Username);
                return Ok(new { Token = token });
            }

            return Unauthorized();
        }

        private string GenerateJwtToken(string username)
        {
            var claims = new[]
            {
                new Claim(ClaimTypes.Name, username)
            };

            var key = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes("ThisIsASecretKeyForJwtToken123456789"));

            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: "MyAuthServer",
                audience: "MyApiUsers",
                claims: claims,
                expires: DateTime.Now.AddMinutes(60),
                signingCredentials: creds);

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

Question 2: Secure an API Endpoint Using JWT

SecureController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace JwtAuthenticationDemo.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   [Authorize]
   public class SecureController : ControllerBase
   {
      [HttpGet("data")]
      public IActionResult GetSecureData()
      {
         return Ok("This is protected data.");
      }
   }
}
```

Question 3: Add Role-Based Authorization

Modify AuthController.cs

Replace

```
var claims = new[]
{
   new Claim(ClaimTypes.Name, username)
};
```

with

```
var claims = new[]
{
   new Claim(ClaimTypes.Name, username),
   new Claim(ClaimTypes.Role, "Admin")
};
```

AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
```

```
namespace JwtAuthenticationDemo.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   [Authorize(Roles = "Admin")]
   public class AdminController : ControllerBase
   {
      [HttpGet("dashboard")]
      public IActionResult GetAdminDashboard()
      {
         return Ok("Welcome to the admin dashboard.");
      }
   }
}
```

Question 4: Validate JWT Token Expiry and Handle Unauthorized Access

Modify Program.cs

Inside AddJwtBearer(), add:

```
options.Events = new JwtBearerEvents
{
   OnAuthenticationFailed = context =>
   {
      if (context.Exception is SecurityTokenExpiredException)
      {
         context.Response.Headers.Add("Token-Expired", "true");
      }

      return Task.CompletedTask;
   }
};
```

Modify AuthController.cs

Replace

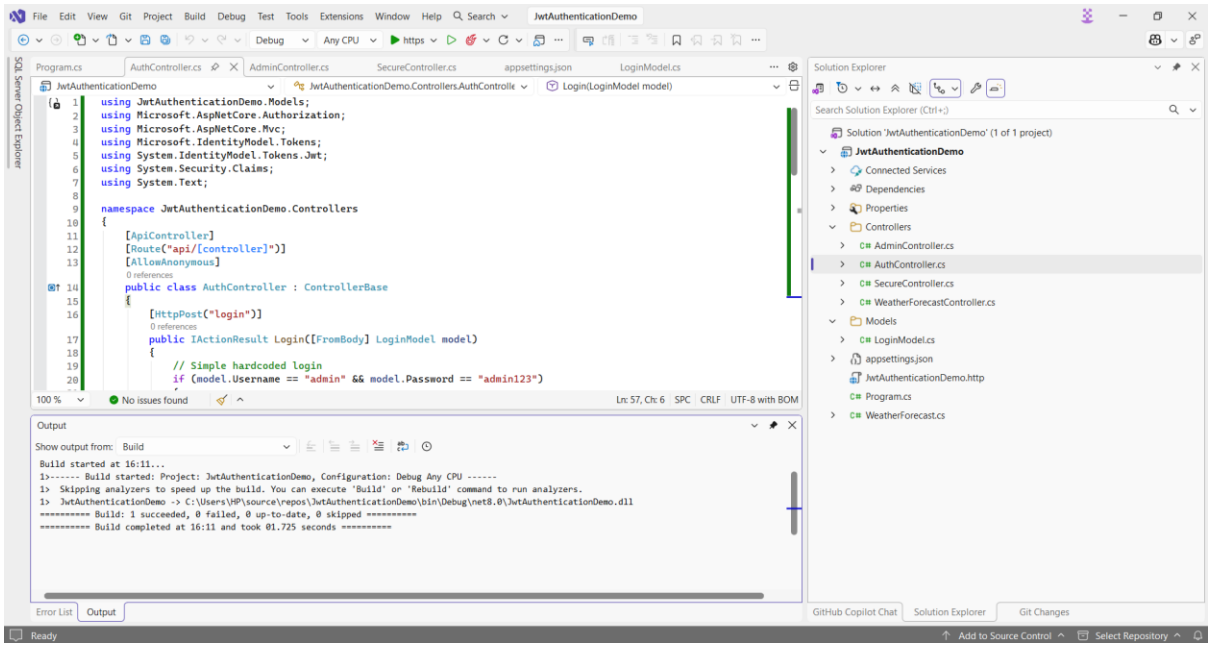
```
expires: DateTime.Now.AddMinutes(60),
```

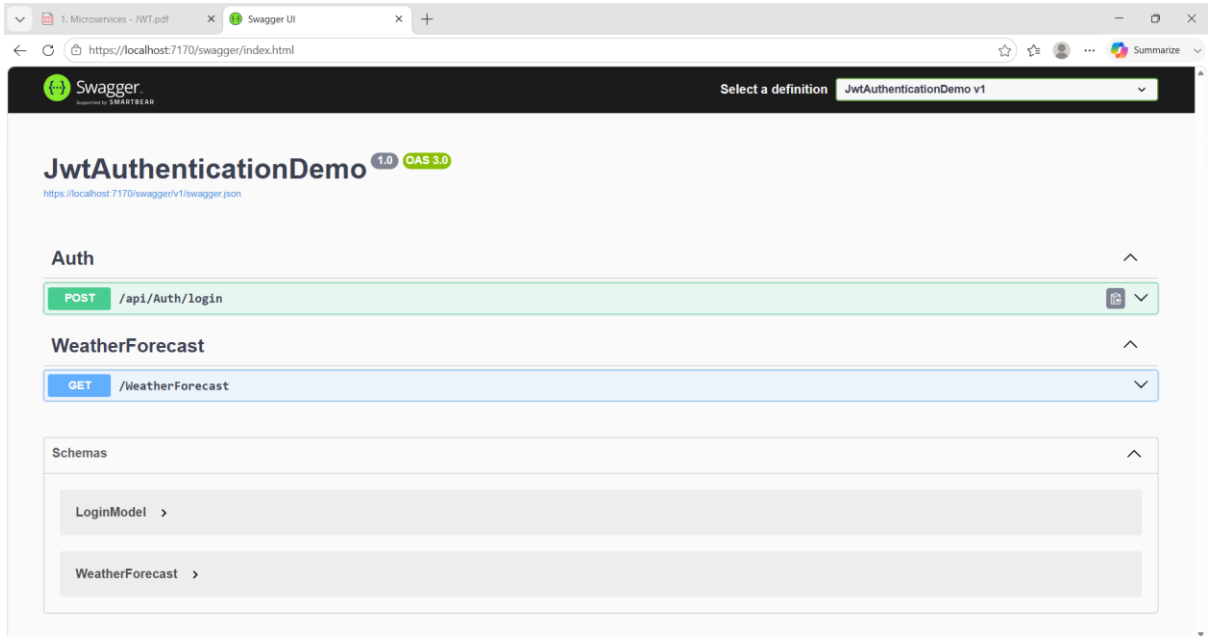
with

```
expires: DateTime.Now.AddMinutes(2),
```

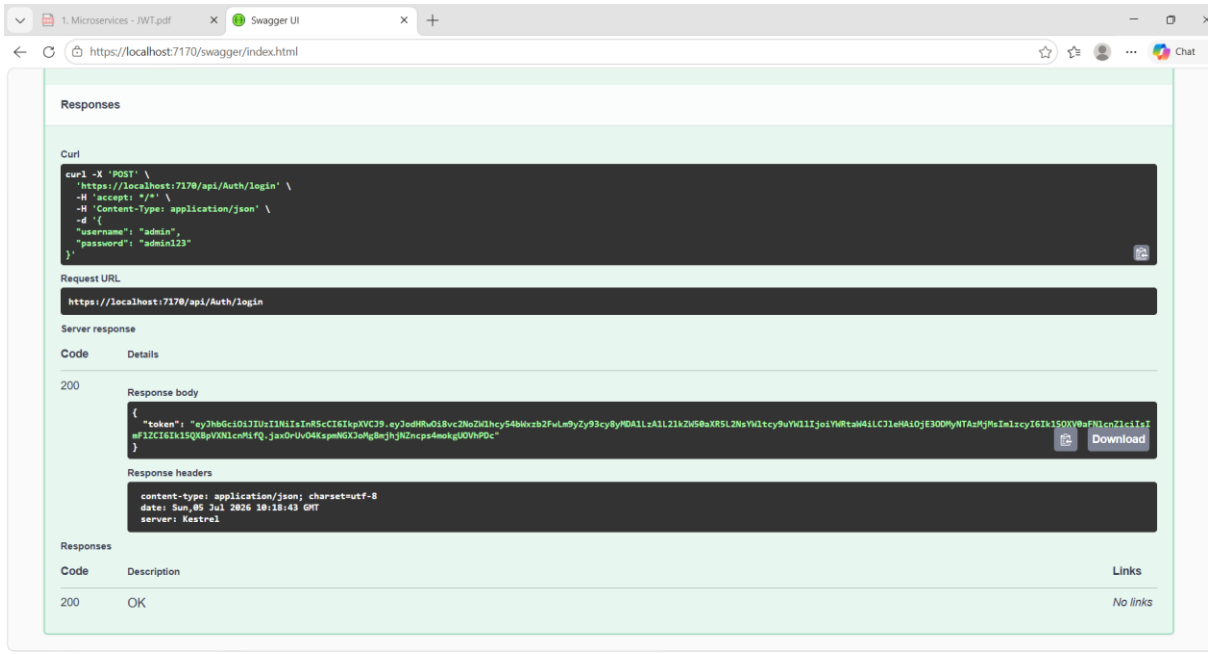
Outputs:

CODES:



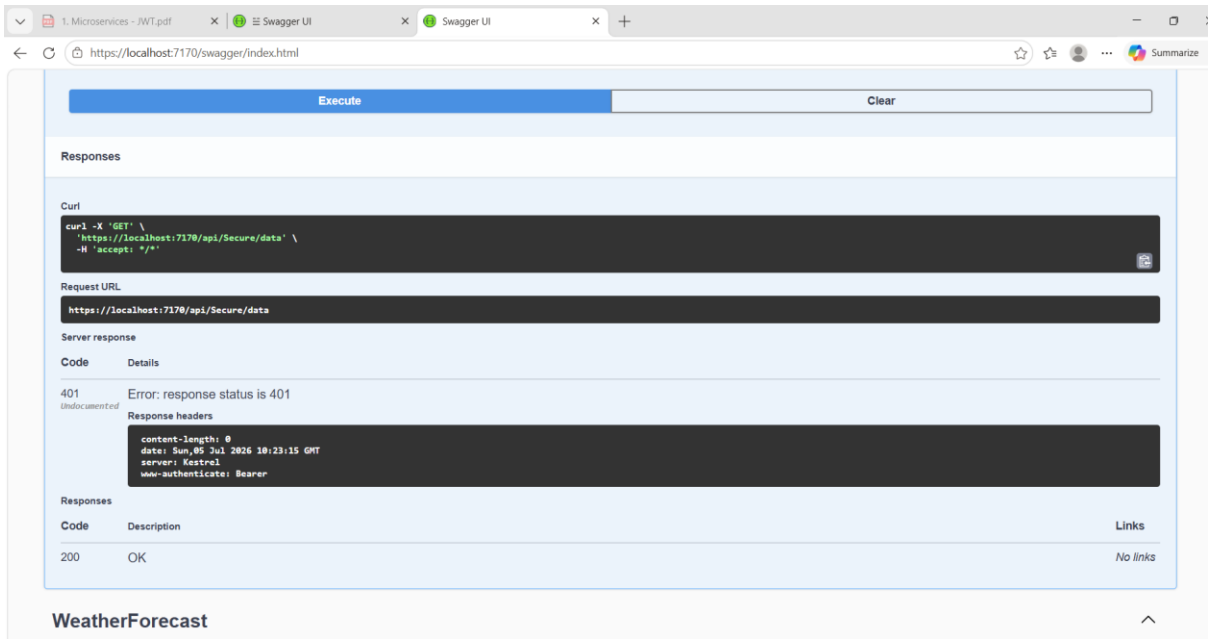


AUTH (POST /api/Auth/login):

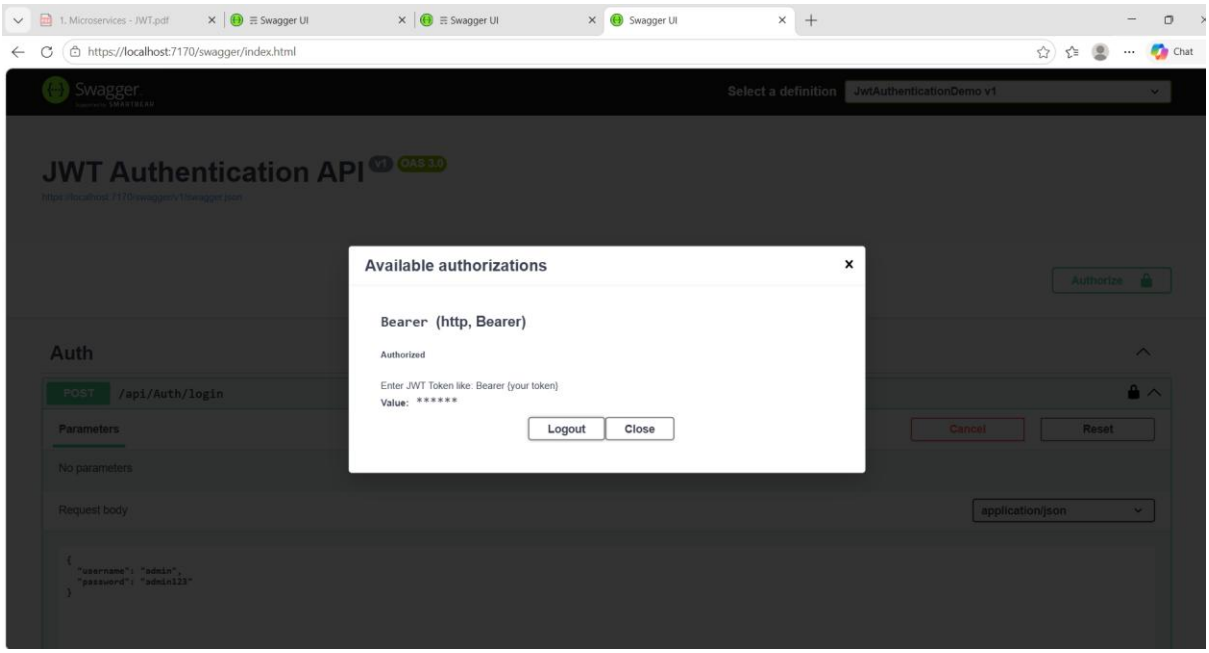


GET /api/Secure/data:

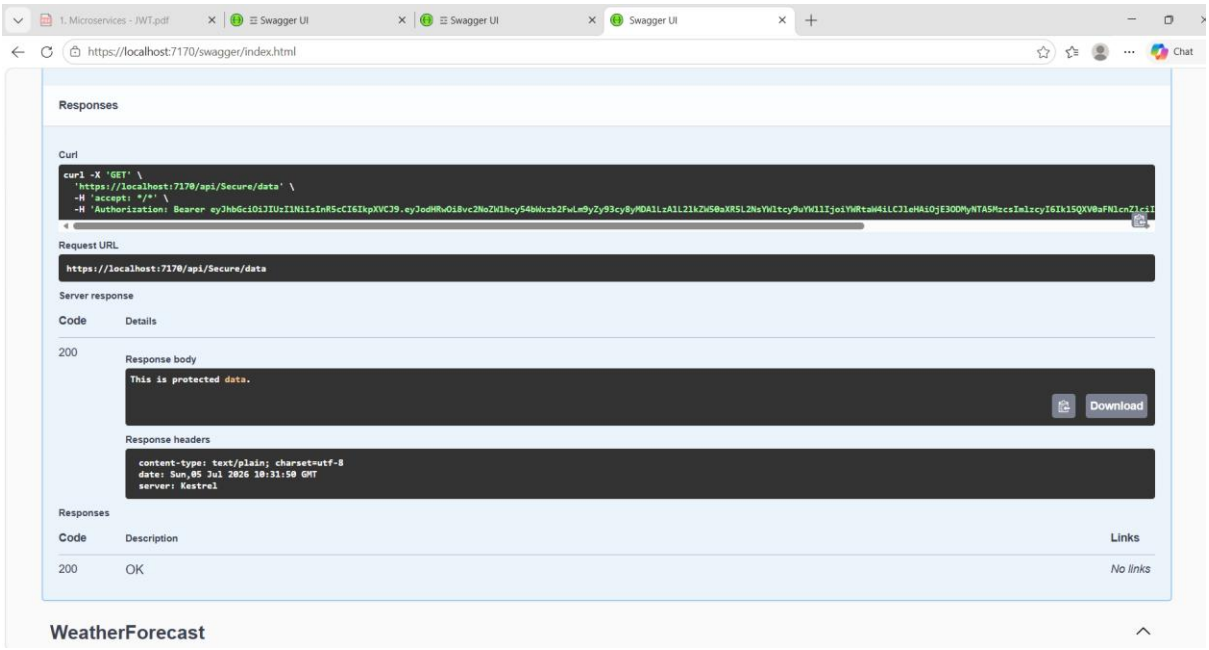
a



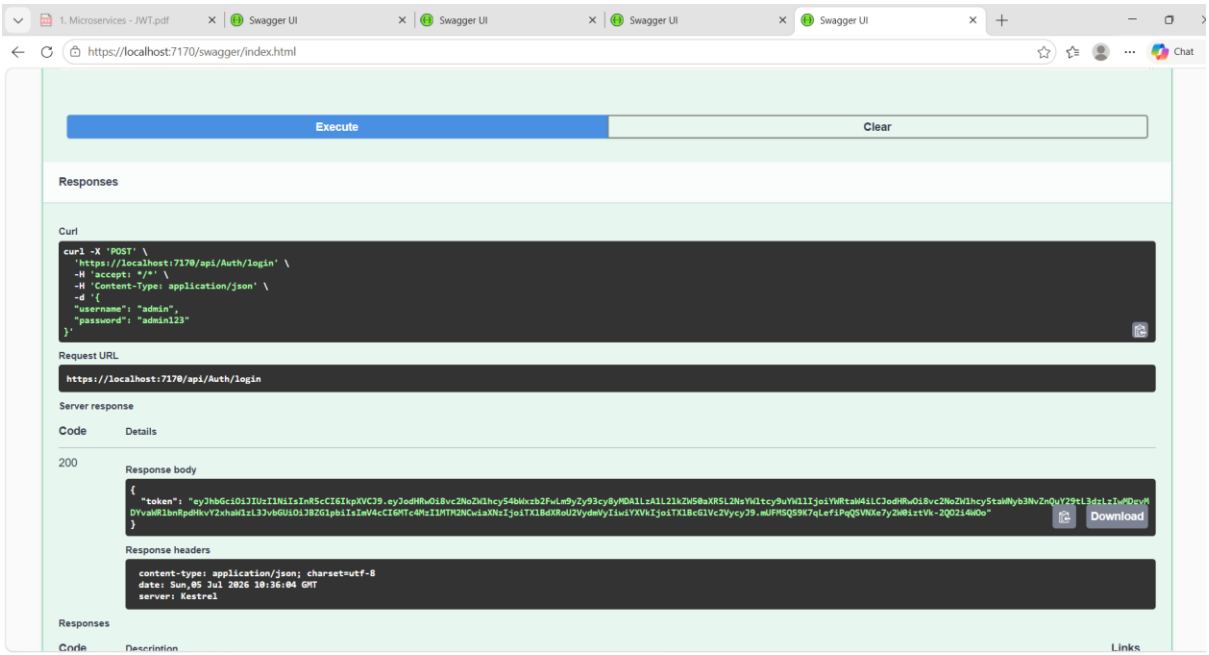
AUTHORIZE:



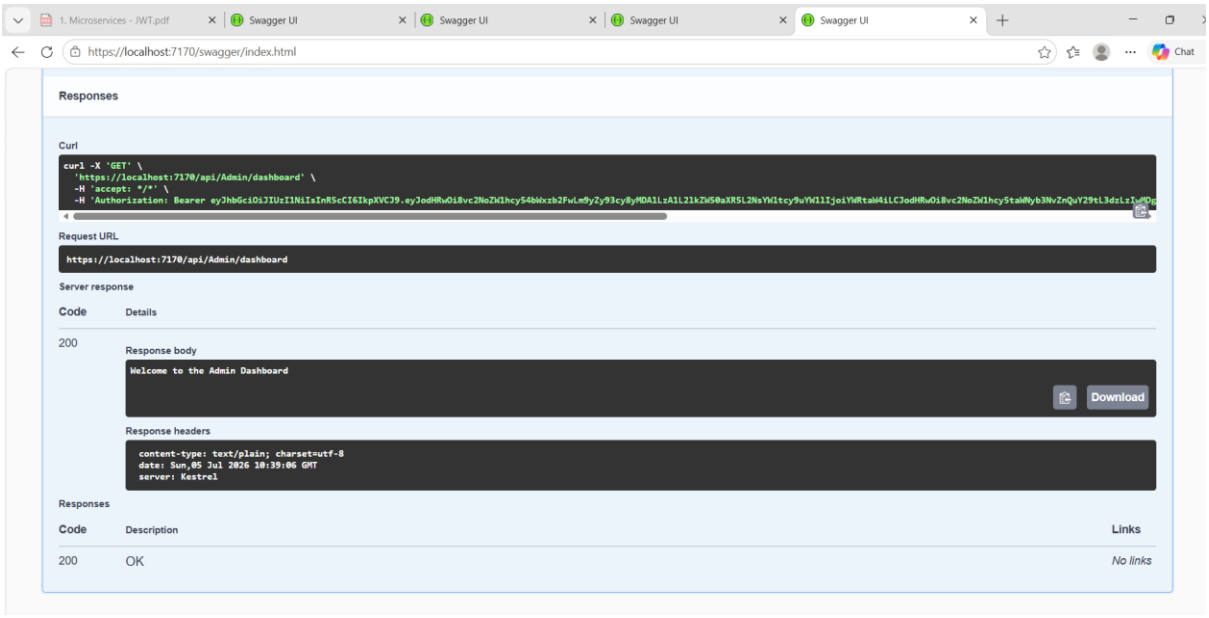
GET /api/Secure/data:

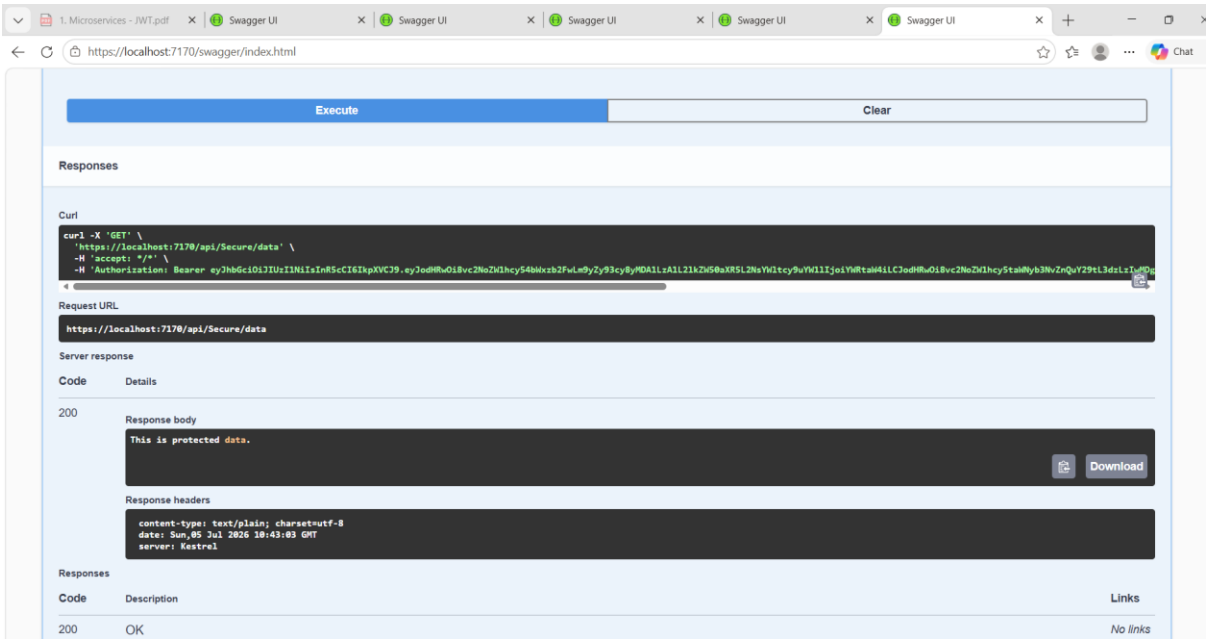


POST /api/Auth/login:



AFTER AUTHORIZING (GET /api/Admin/dashboard):




1. Microservices - JWT.pdf
Swagger UI
Swagger UI
Swagger UI
Swagger UI
Swagger UI
https://localhost:7170/swagger/index.html
Chat
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://localhost:7170/api/Secure/data' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cyBvMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiYWRtaW4iLCJodHRwOi8vc2NoZW1hcy5taWNyb3NvZnQuY29tL3dzLzIwMDg
Request URL
https://localhost:7170/api/Secure/data
Server response
Code
Details
200
Response body
This is protected data.
Download
Response headers
content-type: text/plain; charset=utf-8
date: Sun,05 Jul 2026 10:43:03 GMT
server: Kestrel
Responses
Code
Description
Links
200
OK
No links